Advanced Deep Learning for Image Processing 2026: Assignment 1

Joakim Lindblad, Erik Hallstrom

March 26, 2026

Individual assignment **Due: April 17, 2026**

In this assignment we will perform image segmentation using different encoder-decoder style CNN networks. Use the 'Assignment template' when writing your report (see the Course homepage → Assignments).

Plotting learning curves

In this assignment, you will train multiple neural networks. The process involves the following steps, also know as a training-loop:

1. Utilize *mini-batch gradient descent* to update the network weights. After processing each batch, collect the training loss and accuracy for that batch and append it to two different lists.
2. Upon processing all training batches once, you will have completed one *epoch*.
3. Calculate the average accuracy and loss for the batches within the epoch to determine the epoch's training accuracy and loss using your two lists.
4. Check the model performance on the test set, and calculate the test loss and accuracy for the epoch.

When you have processed all epochs you will have generated *four* lists:

- `train_losses` – contains the train loss for each epoch,
- `test_losses` – contains the test loss for each epoch,
- `train_accuracy` – contains the train accuracy for each epoch,
- `test_accuracy` – contains the test accuracy for each epoch.

Also, measure the elapsed time, in seconds, to record how long your network training took. Record what batch-size and learning rate you used.

When asked about a *learning curves plot*, we'd like you to show a plot similar to Figure 1 in your report. There is a ready-made function you can use for this below:

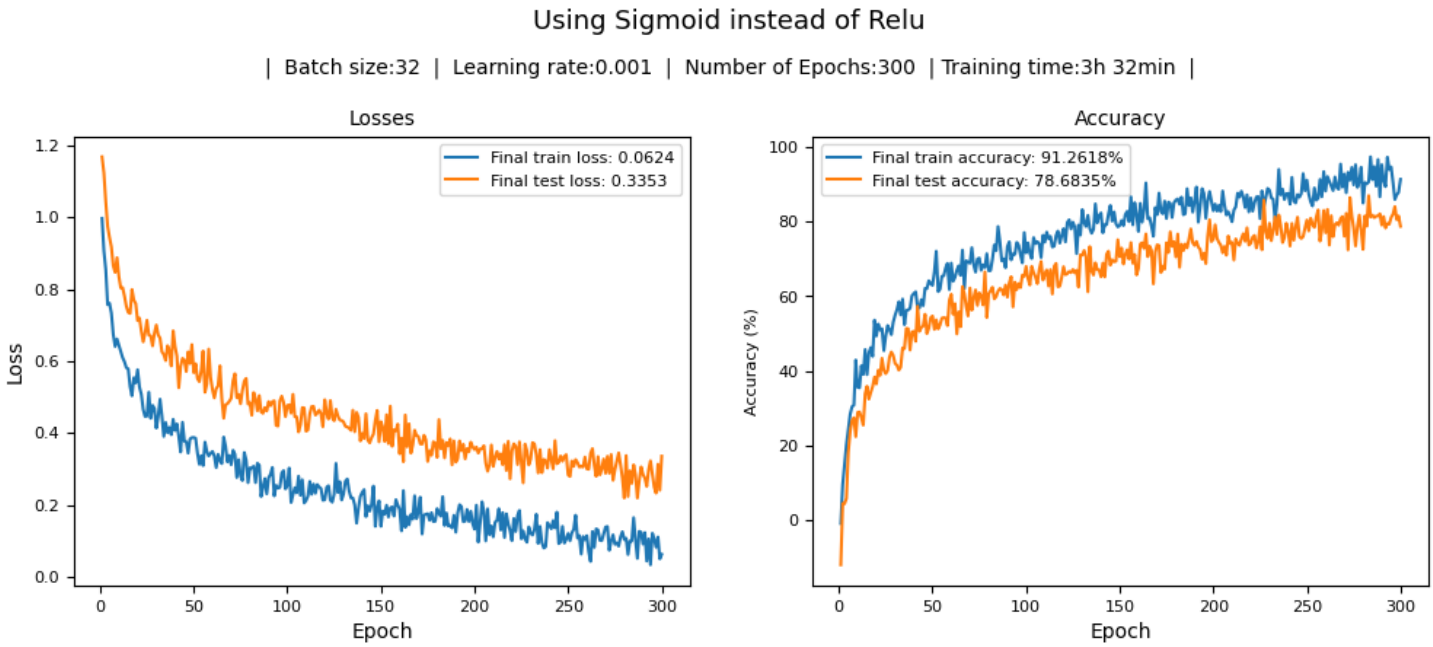


Figure 1: Training curves plot showing loss, accuracy, batch size, learning rate, and training time. We see that the there is a tendency for overfitting (train accuracy clearly above test accuracy) and also that it may be worth trying some 100 more epochs of learning (test accuracy still hints on a growing trend).

You can use the below function (copy-paste) to generate a learning curve plot for an experiment.

```
def learning_curve_plot(title, train_losses, test_losses, train_accuracy, test_accuracy, batch_size, learning_rate, training_time_seconds):
    fsizes = [13, 10, 8]
    fig, axs = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle(title, y=1.1, fontsize=fsizes[0])
    formatted_time = f"{training_time_seconds // 3600}h {(training_time_seconds % 3600) // 60}min"
    sub = f'| Batch size:{batch_size} | Learning rate:{learning_rate} | '
    sub += f'Number of Epochs:{len(train_losses)} | Training time:{formatted_time} |'
    fig.text(0.5, 0.99, sub, ha='center', fontsize=fsizes[1])
    x = range(1, len(train_losses)+1)

    axs[0].plot(x, train_losses, label=f'Final train loss: {train_losses[-1]:.4f}')
    axs[0].plot(x, test_losses, label=f'Final test loss: {test_losses[-1]:.4f}')
    axs[0].set_title('Losses', fontsize=fsizes[1])
    axs[0].set_xlabel('Epoch', fontsize=fsizes[1])
    axs[0].set_ylabel('Loss', fontsize=fsizes[1])
    axs[0].legend(fontsize=fsizes[2])
    axs[0].tick_params(axis='both', labelsize=fsizes[2])
    axs[0].grid()

    axs[1].plot(x, train_accuracy, label=f'Final train accuracy: {train_accuracy[-1]:.4f}%')
    axs[1].plot(x, test_accuracy, label=f'Final test accuracy: {test_accuracy[-1]:.4f}%')
    axs[1].set_title('Accuracy', fontsize=fsizes[1])
    axs[1].set_xlabel('Epoch', fontsize=fsizes[1])
    axs[1].set_ylabel('Accuracy (%)', fontsize=fsizes[2])
    axs[1].legend(fontsize=fsizes[2])
    axs[1].tick_params(axis='both', labelsize=fsizes[2])
    axs[1].grid()
```

Tips

- **Reuse code**: Here you will train a lot of neural networks. Make sure to reuse your code! Write ***functions***, avoid copy-pasting code (that is typically a source of errors, and reduces maintainability of your code).
- **Training time**: If you have access to a GPU it can greatly speed up your training (or Google Colab). However these are very small images. The networks should converge in minutes even if you train on the CPU. If it takes longer you probably have some bug.
- **Debugging**: If something doesn't work, try to isolate the problem. Comment out code that possibly have bugs, do plenty of printouts and plots. Simplify the task until it works and then work from there. Work in small iterations.

1 Semantic segmentation of Biomedical images

*In **semantic image segmentation** the objective is to assign an object class label to each pixel in the image. Such labels may be foreground and background, or different objects as well as object boundaries. Unlike image classification where we are interested in one global label for the entire image, in segmentation we perform classification of each pixel, to get dense pixel-wise label predictions from our model.*

One problem of re-purposing standard CNN architectures, which were developed for classification tasks, for dense predictions are the fully connected layers present in many architectures as we have to preserve the spatial information for the class maps. In 2014, Long et al. introduced Fully Convolutional Networks which allowed dense predictions by avoiding all fully connected layers.

Another problem in using CNNs for segmentation lies in the pooling layers. They increase the field of view, allowing to aggregate context within an image, but discard a lot of the spatial information. In segmentation however, we need an exact alignment of the object class maps and our image at the full spatial resolution since we want to label every pixel in it. One way to overcome this problem is to replace pooling layers altogether and aggregate context from a bigger field of view with special convolutional layers which are often referred to as 'dilated' or 'atrous' convolutions. Another way to handle the effect of pooling is to recover object details by a so-called decoder, which upsamples the information which was encoded in the first part of the CNN. Instead of using simple bilinear interpolation, the interpolation can be learnt by special convolutional layers. There exist different approaches for doing this, transposed convolution (also (incorrectly) known as deconvolution) and pixel shuffle, (a.k.a. sub-pixel convolution) are two commonly used ones. In this assignment we will use transposed convolution.

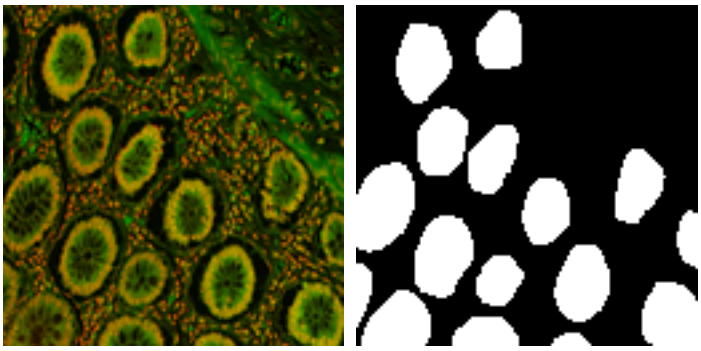
1.1 Segmentation of the Warwick Biomedical Dataset

In this assignment task you will segment images of Hematoxylin and Eosin (H&E) stained microscopy slides of colorectal cancer (see example in Fig. 2). The images show a group of cells that synthesize substances in the human body – so called *glands*. These are important histological structures which are present in most organ systems as the main mechanism for secreting proteins and carbohydrates. The morphology (shape) of glands is used routinely by pathologists to assess the degree of malignancy of several adenocarcinomas, including prostate, breast, lung, and colon cancers.

The dataset (WARWICK.zip) is provided with the task, including ground truth annotations by expert pathologists. To reach improved performance, the stains have been unmixed resulting in two-channel images (the blue component in all the images is constant equal to zero). Your task is to, given an input image, compute an output label-image which has the value one (1) where there is a gland in the image, and zero (0) otherwise.

In order to re-purpose a CNN which was developed for a classification task (e.g. the one used for the MNIST classification task in the Deep Learning course) for (binary) semantic segmentation, a few modifications have to be considered:

- The spatial dimensions of the input, which have been reduced by pooling layers, have to be recovered by transposed convolution layers to gain exact alignment of class maps and the image.
- Do not include any fully-connected layers.



(a) Image 20 (b) Labelling 20

Figure 2: An example image from the Warwick biomedical dataset.

- The softmax has to be taken in the channel-direction to give a class prediction per pixel.
- The loss function has to be adapted to handle a ground truth given by a binary image of the same spatial dimensions as the input image, instead of a single label for the entire image as in classification.
  Tip: You may wish to check out BCEWithLogitsLoss
- You probably want to use the Sigmoid function instead of the Softmax on the final output since since you have only two classes (binary image).
  (It is possible to solve this task using two output layers. Such a solution is similar but not identical – in general it is considered a bit overly redundant. You may wish to take a look at discussions here and here, for example.)

Note, to avoid checkerboard artifacts from the transposed convolutions, it is important that the filter size is an integer times the stride.
Tip: Stride 2 and filter size 4 is usually a good fit.

Exercise 1.1. Semantic segmentation

Design a network for semantic segmentation of the provided data set, e.g., by modifying the network you used for the MNIST classification task in the Deep Learning course.

(a) ***Dice score:*** *Implement a function calculating the dice-score: Sørensen–Dice coefficient. This takes a ground truth binary mask and the tresholded output segmentation mask from the network. You will use this as the final performance metric instead of accuracy (but will not be your loss function):*

$$DSC(A, B) = \frac{2|A \cap B|}{|A| + |B|},$$

for A and B being the gland segmentation maps of the network and the ground truth annotation, respectively.

(b) ***Train the segmentation network:*** *Train the network, provide the learning curve plot supplied above, but replace "Accuracy" with "Dice score". If you did everything correctly you should get a Dice score above 0.75.*

(c) ***Show output****: Visually inspect two of the cases (in the test-set), one where the network preformed well and one where the network did not perform well in terms of dice score, and try to describe what you think makes that particular instance challenging in comparison to the other instances. Include images of these examples in your report (i.e., show side-by-side input image, ground truth mask, and network output).*

Exercise 1.2. Skip connections

Implement skip connections (concatenating activations/feature channels, like in the U-Net paper) in your architecture, bringing higher resolution information into the upsampling path. Evaluate if this improves performance. In particular, check (and visualize in the report) if the use of skip connections leads to better accuracy at the boundary of the glands. Include a learning curve plot above.

Exercise 1.3. Residual connections

Implement residual connections (summing activations, like in the ResNet paper) in your architecture. Evaluate if this improves performance (compare with 1.2). In particular, check if the inclusion of residual connections allows training of deeper networks by replacing each Convolution+activation pair in your architecture with a block of two or three similar pairs, where the residual connection bridges over each such block. Check the speed of learning with and without the residual connections in place (for otherwise identical architectures). Visualize outputs and compare the performance with your previous network architectures in the assignment both quantitatively (DICE score) and qualitatively (visual inspection). Include a learning curve plot above.

Exercise 1.4. Performance optimization, three variations

Now try your best to maximize the performance. Evaluate on the same segmentation task, three different techniques on how to improve the performance on unseen data. Finally, combine the best of the best. Here, you should ***split the training data into training+validation*** *when performing parameter tuning; describe this step explicitly in your report (i.e., how chose to make the split). Suggested is also to make your code deterministic, see.* `https://pytorch.org/docs/stable/notes/randomness.html`*, to reduce fluctuations in your performance evaluation.*

Complement in the report raw performance figures (learning curve plots) with your own subjective opinions regarding the different techniques (e.g. regarding stability and speed of training, design complexity, hyper parameter tuning sensitivity).

For each of these variations answer the following:

- *What did did you change and why?*
- *Provide learning curve plots for losses and DICE score (instead of accuracy).*
- *Is the network overfitting or underfitting?*
- *Provide all hyperparameters to the optimizer, loss function, augmentation etc. (including default values of your chosen framework) such that someone else can try to repeat your work (reproducible research).*
- *Visually inspect two of the cases (in the test-set), one where the network preformed well and one where the network did not perform well in terms of dice score, and try to describe what you think makes that particular instance challenging in comparison to the other instances. Include the examples as images of the network output, ground truth mask and raw image in your report.*

(a) *First variation*

(b) *Second variation*

(c) *Third variation*

(d) *Best of the best*

Submission instructions: Submit a pdf with answers to all exercises. Additionally, submit a zip file with your well commented code. Make sure that it is clear from the (well commented) code how to re-run your experiments for each of the assignment tasks, e.g., a "main" which calls a separate function for each task, 1.1, 1.2a, 1.2b, etc. Always write comments at the beginning of each function indicating what it does, what inputs it takes, and what it returns.

Checklist

Please check these things before handing in:

- ☐ **No miss**: Make sure you didn't miss answering a task: 1.1a, 1.1b, 1.1c, 1.2, 1.3, 1.4a, 1.4b, 1.4c, 1.4d. Is the report containing the required images in Exercise 1.1, 1.2, 1.3, 1.4?
- ☐ **State the problem**: Start the answer to each task with the problem formulation (a short sentence where you write with your own words what you did). For example "Exercise 3: In exercise 3 we implement ... using ... since ..." instead of just "Exercise 3: Answer:..".
- ☐ **Reasonable accuracy**: The Dice score for the segmentation task above 0.75. Tip: Don't forget about data normalization.
- ☐ **Submission format**: Make you sure you format the document with the headings according to the submission format below and that each problem contains the following:

PDF submission format, required titles and items

1 Semantic segmentation

a) Dice score

Show the dice score function in a formatted code block.

b) Segmentation network

Show the learning curve plot

c) Show output

Show two good and two bad samples from test set.

2 Skip connections

Learning curve plot and inspection of a 2-3 test cases.

3 Residual connections

Learning curve plot and inspection of a 2-3 test cases.

4 Performance optimization, three variations

For each variation: learning curve plot and inspection of one good and one bad sample from the test set.

a) First variation: ...

b) Second variation: ...

c) Third variation: ...

You can use 'Assignment template' (on the Course homepage → Assignments), but most importantly follow the headings above.

Information about the WARWICK dataset: The WARWICK dataset, illustrated in Fig. 2, as presented for this assignment consists of 85 training images, and 60 test images, each of size 128×128 and with a binary mask indicating Glandular regions in the images. This is a selected and pre-processed part (colors are unmixed and images are resized) of the Warwick-QU dataset (released with the MICCAI'2015 GlaS contest). See [1] for information about the original data.

References

[1] Korsuk Sirinukunwattana et al. "Gland segmentation in colon histology images: The GlaS challenge contest". In: *Medical Image Analysis* 35 (2017), pp. 489–502. ISSN: 1361-8415. DOI: 10.1016/j.media.2016.08.008.